ISIMS CPC
2025/2026

HIGHER INSTITUTE OF COMPUTER SCIENCE AND MULTIMEDIA OF SFAX

SHINOBI 2.0

TRAINING CONTEST

12 AVRIL

PROBLEM SET

Problem A — The Forbidden Zip File

Time Limit: 1.00 s **Memory Limit:** 256 MB

> *Disaster has struck the ISIMS CPC! The internet has gone down just minutes before the contest is supposed to start. The Head Jury has compiled all the test cases for the contest into a single, heavily encrypted archive named* `test_data.zip`.

The file size of this archive is exactly Z Gigabytes.

You need to physically carry this file from the server room to the main lab. You rummage through your backpack and find two USB flash drives. The first flash drive has exactly X Gigabytes of free space, and the second flash drive has exactly Y Gigabytes of free space.

Because the archive is heavily encrypted, it **cannot be split or extracted** into smaller pieces. The entire `.zip` file must fit completely onto a single flash drive.

Can you save the contest?

Input

The only line of input contains three space-separated integers Z, X, and Y — the size of the zip file, the free space on the first flash drive, and the free space on the second flash drive, respectively.

Output

Print `YES` if you can successfully copy the file onto at least one of your flash drives. Otherwise, print `NO`.

Constraints

- `1 ≤ Z, X, Y ≤ 1000`

Example 1

INPUT

```
10 12 5
```

OUTPUT

```
YES
```

Example 2

INPUT

```
10 6 5
```

OUTPUT

```
NO
```

Note

In the first example, the file is 10 GB. The first flash drive has 12 GB of free space, which is enough to hold the file. The answer is YES.

In the second example, the file is 10 GB. The first drive holds 6 GB and the second holds 5 GB. Even though your total free space is 11 GB, you cannot split the file across the two drives. Neither drive is large enough to hold the 10 GB file on its own. The answer is NO.

Problem B — Fawazir CPC: The Midnight Grind

Time Limit: 1.00 s **Memory Limit:** 256 MB

> *It is the holy month of Ramadan, and the ISIMS CPC is hosting its legendary midnight event: the **Fawazir CPC**! Helmi has prepared a grueling sequence of algorithmic riddles broadcasted live on the projector.*

There are N riddles presented in a strict sequence. The i-th riddle takes exactly A_i seconds of intense focus to solve.

Because the riddles build upon each other conceptually, the rules of the Fawazir state that you must solve a **strictly contiguous sequence** of riddles. You can start participating at any riddle, but once you start, you cannot skip any riddles in between.

It is getting late, and you only have exactly M seconds of brainpower left before you completely crash and need to eat Suhoor. If the total time to solve your chosen sequence strictly exceeds M seconds, you will pass out on your keyboard and get zero points.

Your objective is to carry your team. You must find the maximum number of contiguous riddles you can solve such that their total solving time is less than or equal to M.

Input

The first line contains two integers N and M — the total number of Fawazir riddles and your maximum time limit in seconds.

The second line contains N space-separated integers $A_1, A_2, \ldots, A_N$ — the time required to solve each respective riddle.

Output

Print a single integer: the maximum number of contiguous riddles you can successfully solve.

Constraints

- $1 \le N \le 200{,}000$
- $1 \le M \le 10^{14}$
- $1 \le A_i \le 10^9$

Example

INPUT

```
6 12
3 1 2 7 4 2
```

OUTPUT

```
3
```

Note

In the example, your maximum time limit `M` is 12 seconds.

If we look at the sequence of solve times `[3, 1, 2, 7, 4, 2]`, the optimal contiguous subarray is `[3, 1, 2]` which has a total solve time of 6 (which is ≤ 12) and a length of 3 riddles.

Another valid sequence of length 3 is `[1, 2, 7]` which has a sum of 10.
You cannot take `[3, 1, 2, 7]` because the sum is 13, which exceeds your time limit of 12.
The answer is the maximum length found, which is 3.

Problem C — Scooby & Dora: The Cursed Snack

Time Limit: 1.00 s **Memory Limit:** 256 MB

> *"Ruh-roh, Raggy!" Scooby-Doo and Dora the Explorer have teamed up to unlock the ancient Temple of Mysteries. To open the giant stone door, they need to insert a sequence of magical Scooby Snacks into the temple's altar.*

They have exactly N magical Scooby Snacks. The i-th snack has a "Mystery Power" of exactly A_i.

According to the map, the temple's door will only open if the **overall resonance** of the altar is powerful enough. The resonance is defined as the *Greatest Common Divisor (GCD)* of all the inserted snacks.

However, Boots the Monkey just noticed something terrible: *Swiper the Fox* snuck into their backpacks and replaced exactly one good Scooby Snack with a cursed, dud snack! This cursed snack is completely ruining their GCD resonance.

Dora needs your help! Can you find the cursed snack? You must throw away **exactly one** Scooby Snack so that the resonance (the GCD of the remaining $N - 1$ snacks) is as large as possible!

Input

The first line contains a single integer N — the number of Scooby Snacks they currently have.

The second line contains N space-separated integers $A_1, A_2, \ldots, A_N$ — the Mystery Power of each snack.

Output

Print a single integer: the maximum possible resonance (GCD) of the altar after throwing away exactly one cursed Scooby Snack.

Constraints

- $2 \le N \le 200{,}000$
- $1 \le A_i \le 10^9$

Example

INPUT

```
4
12 15 18 24
```

OUTPUT

```
6
```

Note

In the example, the Mystery Powers are `[12, 15, 18, 24]`.

- If Dora throws away 12, the remaining GCD is `gcd(15, 18, 24) = 3`.
- If Dora throws away 15, the remaining GCD is `gcd(12, 18, 24) = 6`.
- If Dora throws away 18, the remaining GCD is `gcd(12, 15, 24) = 3`.
- If Dora throws away 24, the remaining GCD is `gcd(12, 15, 18) = 3`.

The maximum possible resonance after throwing one away is 6. Thus, Swiper's cursed snack was clearly the one with power 15!

Problem D — SpongeBob's Secret Sauce

Time Limit: 1.50 s **Memory Limit:** 256 MB

> *"Order up, Squidward!" The Krusty Krab has just received the largest catering order in Bikini Bottom history. SpongeBob has lined up exactly* N *Krabby Patties on his massive grill.*

Initially, every single Krabby Patty has exactly 0 drops of Plankton's stolen... I mean, Mr. Krabs' Secret Sauce on it.

SpongeBob moves back and forth across the grill in a frenzy. He performs Q rapid-fire actions. For each action, he takes his sauce bottle and squirts exactly V drops of sauce on every single patty starting from index L all the way to index R (inclusive).

After SpongeBob finishes all Q of his chaotic grilling actions, Mr. Krabs comes out to inspect the order. He wants a complete report on the exact amount of sauce on every single patty before they are served to the customers.

Because the grill is so massive, simulating SpongeBob's actions one by one will take way too much time, and the customers will get angry! You need to find a smarter way to calculate the final state of the grill.

Input

The first line contains two integers N and Q — the number of Krabby Patties on the grill and the number of SpongeBob's sauce actions.

The next Q lines each contain three space-separated integers L, R, and V — representing an action where SpongeBob adds V drops of sauce to all patties from index L to R.
Note: The indices are 1-based.

Output

Print a single line containing `N` space-separated integers, representing the final amount of Secret Sauce on each Krabby Patty from index 1 to `N`.

Constraints

- `1 ≤ N, Q ≤ 200,000`
- `1 ≤ L ≤ R ≤ N`
- `1 ≤ V ≤ 10,000`

Example

INPUT

```
5 3
1 3 2
2 5 1
4 4 5
```

OUTPUT

```
2 3 3 6 1
```

Note

Let's trace SpongeBob's actions on the 5 patties:

- **Initial state:** `[0, 0, 0, 0, 0]`
- **Action 1 (1 3 2):** Add 2 drops from patty 1 to 3.
  State becomes: `[2, 2, 2, 0, 0]`
- **Action 2 (2 5 1):** Add 1 drop from patty 2 to 5.
  State becomes: `[2, 3, 3, 1, 1]`
- **Action 3 (4 4 5):** Add 5 drops to patty 4 only.
  State becomes: `[2, 3, 3, 6, 1]`

The final output is exactly `2 3 3 6 1`.

Problem E — Operation Bizerte: Frontline Reinforcements

Time Limit: 1.00 s **Memory Limit:** 256 MB

> *The northern naval base at Bizerte is bracing for a massive amphibious assault. As the lead tactical commander of the ISIMS defense forces, you oversee* N *distinct coastal bunkers.*

Currently, the i-th bunker is manned by A_i active operatives. However, due to structural limits, the i-th bunker has a maximum physical capacity of B_i operatives. It cannot hold more than this number.

The capital has just authorized the deployment of K elite reserve operatives to your command. You can distribute these reserves among your bunkers however you see fit, provided no bunker exceeds its maximum capacity B_i.

Your tactical doctrine dictates that the defense line is only as strong as its weakest link. Therefore, your objective is to distribute the reserves to **maximize the size of the smallest garrison** across all bunkers.

Input

The first line contains two integers N and K — the number of coastal bunkers and the total number of reserve operatives available.

The second line contains N space-separated integers $A_1, A_2, \ldots, A_N$ — the current number of operatives in each bunker.

The third line contains N space-separated integers $B_1, B_2, \ldots, B_N$ — the maximum capacity of each bunker.

Output

Print a single integer: the maximum possible size of the smallest garrison after optimally distributing the `K` reserves.

Constraints

- $1 \le N \le 200,000$
- $1 \le K \le 10^{14}$
- $0 \le A_i \le B_i \le 10^9$

Example

INPUT

```
3 4
1 2 3
4 4 3
```

OUTPUT

```
3
```

Note

In the example, you have 3 bunkers and 4 reserves to distribute.

- Bunker 1: Currently 1 operative, Max capacity 4.
- Bunker 2: Currently 2 operatives, Max capacity 4.
- Bunker 3: Currently 3 operatives, Max capacity 3.

Notice that Bunker 3 is already at its maximum capacity and cannot receive any reinforcements. Its size will remain 3. You can send 2 reserves to Bunker 1 (making it size 3) and 1 reserve to Bunker 2 (making it size 3). You have used 3 out of your 4 available reserves. All bunkers now have at least 3 operatives. You cannot make the minimum size 4, because Bunker 3 cannot hold 4 operatives. Therefore, the answer is 3.